

Virtualización de servicios web con Vagrant

Debido al alto coste computacional de las máquinas virtuales basadas en tecnologías como VirtualBox, este tipo de esquemas de virtualización solamente se aplican a determinados servicios. Uno de los más comunes son los servicios web, debido a que multitud de usuarios (muchos sin conocimientos específicos sobre la gestión de redes) desean emplearlos. También podemos encontrar este tipo de aplicaciones en empresas de desarrollo de software, donde el primer paso consiste en la creación de un entorno de desarrollo que pueda ser replicable de manera sencilla y rápida.

En esta práctica, pedimos crear un script Vagrant (Ruby) para desplegar una máquina virtual Linux (se podrá emplear la distribución que se prefiera), donde se ejecute un servidor web Node.js que debe ser accesible tanto desde la propia máquina virtual, como desde el exterior. Para ello, el script que desarrolle deberá contener todas las sentencias necesarias para instalar el framework de Node, descargar de GitHub (o cualquier otro repositorio Git) el código del servidor y ejecutarlo. Recuerde que utilizando Vagrant es posible ejecutar comandos en la Bash en el momento del despliegue.

Como servidor web puede emplear el disponible en el siguiente enlace, conocido de asignaturas previas como “Seguridad en sistemas y redes” o “Redes avanzadas”:

<https://github.com/gisai/SSR-master-server>

Si lo desea puede emplear cualquier otro servidor que usted desarrolle.

Debe aplicar redirección de puertos, para que el servidor sea accesible desde el exterior en el puerto 8080, pero se accesible desde la propia máquina virtual en el puerto 3000. Asegúrese también de asignar a la máquina virtual una dirección de red, acorde al tipo de conexión que haya escogido (pública o privada).

Como resultado de la práctica se deberá entregar el correspondiente escrito en Ruby (Vagrant).

Se valorará también que los alumnos exploren configuraciones adicionales a las mínimas requeridas, y que experimenten con esquemas de red avanzados (como por ejemplo que una misma máquina virtual disponga de diversas interfaces).

Estudie el consumo en memoria y CPU de este servicio virtualizado, y discuta cuánto consumiría el despliegue de una red virtualizada que replicase la infraestructura del laboratorio (cuyo dibujo encuentra en Moodle). Puede usar el comando `watch` de Linux para vigilar la manera en que las imágenes crecen según continúa la operativa de la red.

```
watch "ls -al [dirección de la imagen en el sistema de  
ficheros]"
```

Si trabaja en un sistema Windows, lo más sencillo es que escriba un fichero `watch.bat` que le permita “imitar” el comportamiento del comando `watch`. Por ejemplo:

```
@ECHO OFF

:loop

cls

%*

timeout /t 5 > NUL

goto loop
```

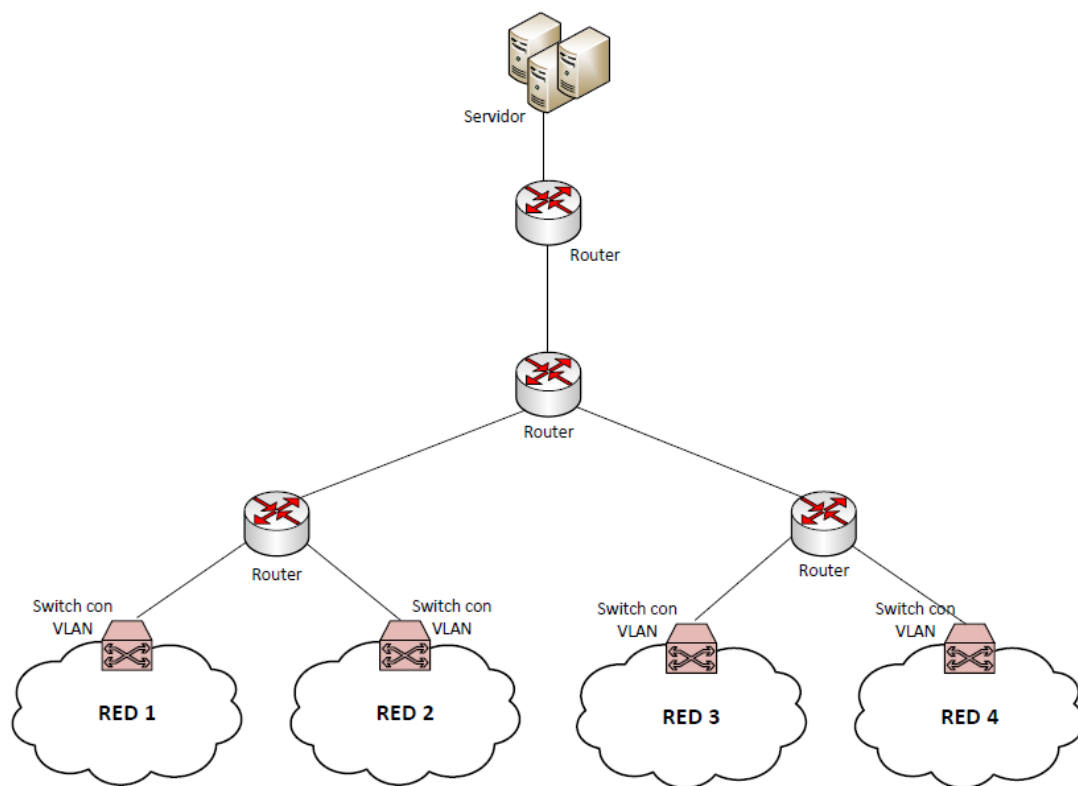
Y luego ejecute en el directorio donde se almacenan las máquinas virtuales, usando SIMBOLO DEL SISTEMA, el comando `watch dir/s`

Se valorará positivamente que compare los resultados obtenidos con la capacidad de los servidores comerciales de la actualidad.

Si lo desea puede entregar también un documento escrito donde explique las pruebas realizadas y las configuraciones que proporciona mediante su script.

Virtualización de infraestructura de red con KVM

Se pide desarrollar un script que despliegue una red virtualizada, según el dibujo mostrado a continuación (refleja la infraestructura montada en el laboratorio 3103) y empleando la tecnología QEMU-KVM:



Puede emplear el lenguaje que se prefiera, aunque aconsejamos el uso de Python y Bash por disponer de librerías que facilitan la implementación. El script correspondiente deberá generar los ficheros XML necesarios para desplegar todas las máquinas virtuales. Para evitar la multiplicación innecesaria de las máquinas virtuales, las cuatro subredes reflejadas en el dibujo pueden consistir en un único PC (qué puede ser incluso el anfitrión si se desea).

CONSEJO: Para crear y modificar los ficheros XML de definición de las MVs, se utilizará alguna de las librerías disponibles en Python para leer y modificar ficheros XML. Se recomienda utilizar la librería `lxml.etree` (<http://lxml.de>). Pero puede optarse por cualquier otra.

El script deberá no solamente desplegar todas las máquinas, sino además instalar la Software Routing Suite en aquellas máquinas que actúen como router, y configurar las rutas estáticas correspondientes para lograr conectividad global. En el servidor se deberá desplegar un servicio web, para el que puede reciclar el código que empleó en el apartado anterior.

El objetivo final de la infraestructura es lograr que cualquiera de las máquinas situadas en la parte inferior de la red tenga acceso al servicio web del servidor. La máquina anfitriona también deberá tener acceso al servidor, en el caso de que ésta no forme parte ya de una de las subredes de la parte inferior.

Como resultado de la práctica, deberá entregar el script correspondiente, además de una memoria explicando el plan de numeración que ha dispuesto para la infraestructura (haga un dibujo), así como las tablas de enrutamiento que ha configurado en los distintos router. Deberá también indicar el plan de pruebas que ha llevado a cabo para confirmar que existe conectividad global.

Se valorará que los alumnos apliquen configuraciones adicionales a las mínimas requeridas en cualquiera de las máquinas virtuales, así como en el Software Routing Suite.

PROPUESTA DE SOLUCIÓN:

El uso de tecnologías a nivel de Kernel en el sistema operativo, hace que emplear de KVM y libvirt sea muy sensible a la distribución y versión del sistema que tengamos tanto en la máquina anfitriona como en la máquina virtual. Por ello **aconsejamos encarecidamente** que para resolver esta práctica y generar la red virtual descrita anteriormente, se emplee como base la **imagen de disco y el fichero XML que se puede encontrar en Moodle**.

En dicha imagen de disco, el superusuario es `root`, con password: `agr`.

Si lo desea, puede cambiar el hostname y la contraseña.

Aconsejamos que el PC anfitrión se conecte a uno de los bridge desde las subredes inferiores (RED 1, 2, 3 y 4), para que pueda comprobar el flujo de tráfico hacia el exterior.

Es posible emplear cualquier otra distribución o imagen, pero puede que sea necesario enfrentar problemas que requieran amplios conocimientos del funcionamiento interno del sistema operativo.

RECOMENDACIONES:

Los mensajes que se quiera que el programa siempre muestre en pantalla se pueden imprimir utilizando la función `print/printf`. Por el contrario, para la depuración del script es útil que el programa muestre trazas de información en pantalla que se puedan eliminar fácilmente una vez se haya depurado el programa. Se puede utilizar el módulo “logging” de Python.

Para la gestión de las máquinas virtuales (arranque, parada, etc.) se recomienda utilizar el comando “`sudo virsh define|start|shutdown|destroy`” explicado en los materiales.

Si desea arrancar terminales desde el script para las distintas máquinas virtuales, puede emplear el comando `xterm`. Por ejemplo:

```
xterm -e "sudo virsh console miMaquina"
```

Tenga en cuenta que los comandos anteriores deben ser ejecutarlos en background para que el script no se quede detenido cuando se ejecuta el comando (añada un "&" al final del comando para lograrlo).

Los comandos `sudo virt-cat` y `sudo virt-ls` pueden ayudar a listar los ficheros o volcar su contenido sin necesidad de acceder a la máquina virtual.